

Tutorial 6: Noughts and crosses

In this tutorial we will implement the game **noughts and crosses** (or **tic-tac-toe**). Guidance is available after the instructions, but first, try to do the assignment without it. Analyse the problem, design your own data types, and break up the problem into logical functions. For unexpected inputs, you may throw exceptions (crashing the game).

Exercise 1: Implement the game noughts and crosses, where the computer opponent plays random moves. It should look (roughly) as follows (player is X and opponent is O):

```
ghci> main
```

```

  |  |
---+---+---
  |  |
---+---+---
  |  |

```

Choose your move from 1 2 3 4 5 6 7 8 9: 5

```

  |  |
---+---+---
  | X |
---+---+---
  |  |

```

```

  |  |
---+---+---
  | X |
---+---+---
O |  |

```

Choose your move from 1 2 3 4 6 8 9: 9

```

  |  |
---+---+---
  | X |
---+---+---
O |  | X

```

```

  |   | 0
---+---+---
  | X |
---+---+---
0 |   | X

```

Choose your move from 1 2 4 6 8: 1

```

X |   | 0
---+---+---
  | X |
---+---+---
0 |   | X

```

You won!

Extra: implement the following features:

- Add an introductory text that shows which number corresponds to which move.
- At the start of the game, for each player X and O, offer the selection between a human-controlled player or a random computer.

Challenge: implement a computer player that plays optimally, in the following sense:

- if there are **winning** moves, i.e. that guarantee a win, it chooses one; otherwise,
- if there are **non-losing** moves, i.e. that guarantee at least a draw, it chooses one; otherwise,
- it chooses a random move.

Note that such a strategy does not have to behave predictably: for example, it doesn't need to choose an immediate winning move (one that creates a three-in-a-row), if other moves guarantee a later win as well. For further reading, look up **minimax** strategies.

```

ghci> main
Welcome to noughts and crosses
Choose your moves from:

```

```

1 | 2 | 3
---+---+---
4 | 5 | 6
---+---+---
7 | 8 | 9

```

Choose Player X (0: human, 1: random, 2: optimal): 2

Choose Player O (0: human, 1: random, 2: optimal): 1

```

  |  |
---+---+---
  | X |
---+---+---
  |  |

```

```

  |  |
---+---+---
  | X | O
---+---+---
  |  |

```

```

  |  | X
---+---+---
  | X | O
---+---+---
  |  |

```

```

O |  | X
---+---+---
  | X | O
---+---+---
  |  |

```

```

O |  | X
---+---+---
  | X | O
---+---+---
X |  |

```

Player X won!

Guidance

Please try to complete the exercise without using this guidance at first, until you are stuck, or you are finished but want to improve your solution.

Data types The first thing to do is to set up your data types. We need to represent:

- the two players
- the game board (the nine squares)
- squares on the board (empty or belonging to either player)
- moves (selecting one of the nine squares)

Create a data type for each. This can be done as **type** aliases for existing types (`Bool`, `Int`, `Maybe`, `Either`, lists, and tuples) or as new **data** types. There is a trade-off between safety and convenience here: type aliases are more convenient, but may not hold the exact data required; new data types can be tailor-made for this problem, but are more cumbersome and may require more auxiliary functions to be defined. You are allowed to choose as you prefer, though you are encouraged to consider what both options would look like, and what advantages either may have.

The board Next, create the functions needed for displaying and manipulating the board. A natural start is `showBoard` which takes a board to a `String` (use newlines `'\n'`). Try to create auxiliary functions for repeat functionality. We then need the following functionality:

- to find the available moves on a board;
- to decide if a given player has won;
- to carry out a move by a player.

The game loop Finally, we need to create the main game loop. This should consist of a function `main :: IO ()` which initializes the game with an empty board and calls a `loop` function, the game loop itself, which should carry the data needed for the game to progress (the game state) as its inputs.

There are two ways of breaking up the game loop into steps. One is to take a step as a move by a single player, `X` or `O`, and the game state records whose turn it is. The other is to take a step as two moves, the first by `X` and the second by `O`, checking win conditions in between. Either way it is useful to separate out the functionality of a move by `X`, including taking input from the player, and that of a move by `O`, including generating a random value, into separate functions.